

Inventory Tracker/Recipe Book - Group 3

Liam McKenzie
Kat Toolan
Nick Martin
Nathan Oden
Jacob Choi

A. Application Overview

The Inventory Tracker and Recipe Book is a web-based database application designed to help people manage the food in their kitchen more effectively. The application gives users a centralized place to track what ingredients they have on hand, when those ingredients expire, which recipes they want to cook, and what they need to pick up at the store.

The problem the application addresses is straightforward: people frequently forget what is in their kitchen, let food expire before using it, and struggle to connect what they have to what they can cook. This leads to wasted food and unnecessary grocery spending. By combining inventory tracking with a recipe system and a grocery list, the application brings these tasks together in one place.

The target audience is anyone who regularly cooks at home and wants a better way to stay organized, particularly busy people who do not always have time to take stock of their kitchen before shopping or planning meals. The application does not require any prior technical knowledge and is designed to be approachable for general users.

The key features that make the application useful include per-user inventory management with expiration date tracking, a recipe library with tagging and favoriting, a grocery list that tracks what has and has not been purchased, nutritional information stored for ingredients and recipes, and integration with the Spoonacular API to allow users to pull in external recipes. Together these features give users a complete picture of their kitchen and make it easier to plan meals and reduce food waste.

B. Database Design

The database is built around ten tables that together support all of the application's core functionality. The central entities are users, ingredients, and recipes, with supporting tables handling the relationships between them and storing additional data such as nutritional values, tags, and grocery list entries.

The Users table holds each account with a `user_id` as the primary key alongside the user's name and a hashed password. Users are the root of most relationships in the database, since inventory, favorites, and grocery lists are all scoped to individual accounts.

The Ingredients table stores a catalog of ingredients with an ingredient_id primary key, ingredient name, and a default unit of measurement. This table acts as a reference that both the Inventory and Recipe_Ingredients tables point to, avoiding duplication of ingredient definitions.

The Inventory table tracks what a specific user has on hand. It references both user_id and ingredient_id as foreign keys and adds quantity, unit, expiration date, and date opened. This is the primary table for the expiration tracking feature.

The Recipes table holds recipe records with a recipe_id primary key and a foreign key to user_id indicating who saved it. It stores the recipe name, instructions, an image URL, the source API if applicable, a cache priority enum indicating whether the recipe is a favorite, recently used, or temporary, and serving size.

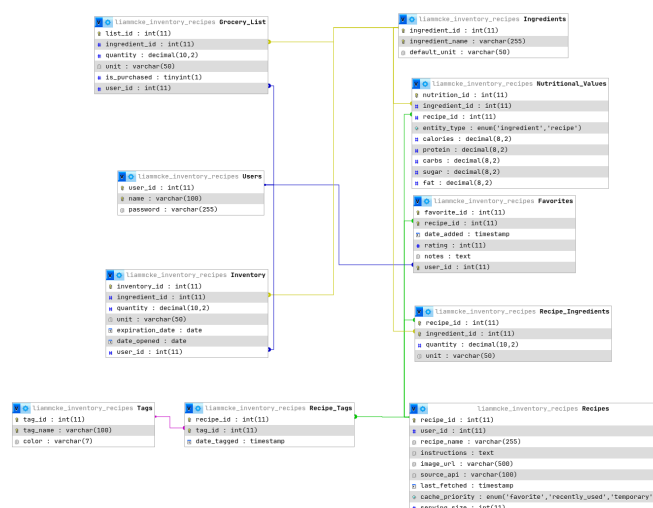
The Recipe_Ingredients table is a junction table linking recipes to their ingredient requirements. It uses recipe_id and ingredient_id as a composite primary key and adds quantity and unit for each ingredient in that recipe.

The Favorites table allows users to save recipes they like. It links user_id and recipe_id and adds a timestamp for when the recipe was favorited, a numeric rating, and optional notes.

The Tags table defines a catalog of tags with a tag_id, tag name, and an associated color. The Recipe_Tags table is a junction linking recipes to tags, with a timestamp recording when the tag was applied.

The Grocery_List table holds individual items a user intends to purchase. It links to user_id and ingredient_id and stores quantity, unit, and a boolean flag indicating whether the item has been purchased.

The Nutritional_Values table stores macronutrient data for both ingredients and recipes. It uses an entity_type enum to distinguish between the two and references either ingredient_id or recipe_id depending on the type. It stores calories, protein, carbohydrates, sugar, and fat as decimal values.



The design supports the application's functionality by keeping user data isolated through foreign key relationships to the Users table, centralizing ingredient definitions to avoid redundancy, and using junction tables to handle many-to-many relationships cleanly between recipes, ingredients, and tags.

C. Database Implementation

The database was implemented in MySQL and is hosted through cPanel. Each table was created with explicit primary keys, foreign key constraints, and data type definitions appropriate to the data being stored.

The Users table uses `user_id` as an auto-incrementing primary key. Passwords are stored as `varchar(255)` to accommodate hashed values. The Ingredients table uses `ingredient_id` as its primary key and stores `ingredient_name` as `varchar(255)` with a default `_unit` of `varchar(50)`. The Inventory table uses `inventory_id` as its primary key and enforces foreign keys on both `ingredient_id` referencing Ingredients and `user_id` referencing Users. Expiration and open dates are stored as DATE types.

The Recipes table uses `recipe_id` as its primary key with a foreign key on `user_id`. The `cache_priority` column uses an ENUM restricted to the values `favorite`, `recently_used`, and `temporary`, which controls how cached recipe data is managed. The Recipe_Ingredients table uses a composite primary key on `recipe_id` and `ingredient_id`, both of which are foreign keys to their respective tables.

The Favorites table uses `favorite_id` as its primary key with foreign keys on both `recipe_id` and `user_id`. The `rating` column is stored as INT. The Tags table uses `tag_id` as its primary key and stores `color` as `varchar(7)` to hold hex color codes. Recipe_Tags uses a composite primary key on `recipe_id` and `tag_id`.

The Grocery_List table uses `list_id` as its primary key with foreign keys on `ingredient_id` and `user_id`. The `is_purchased` flag is stored as TINYINT(1), functioning as a boolean. The Nutritional_Values table uses `nutrition_id` as its primary key and uses an ENUM on `entity_type` to distinguish between ingredient and recipe records.

The database was populated primarily with synthetic data covering a range of ingredients, recipes, nutritional values, tags, and user accounts. Additional recipe data was appended during live demonstration and testing using the Spoonacular API integration, which allows recipe records to be fetched and stored in the Recipes and Recipe_Ingredients tables dynamically.

Favorites Table:

 Table structure		 Relation view								
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action	
☐	1 favorite_id 	int(11)			No	None		AUTO_INCREMENT	 Change	 Drop  More
☐	2 recipe_id 	int(11)			No	None			 Change	 Drop  More
☐	3 date_added	timestamp			No	CURRENT_TIMESTAMP			 Change	 Drop  More
☐	4 rating	int(11)			Yes	NULL			 Change	 Drop  More
☐	5 notes	text	utf8_unicode_ci		Yes	NULL			 Change	 Drop  More
☐	6 user_id 	int(11)			Yes	NULL			 Change	 Drop  More

Grocery_List Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action	
☐	1 list_id 	int(11)			No	None		AUTO_INCREMENT	 Change	 Drop  More
☐	2 ingredient_id 	int(11)			No	None			 Change	 Drop  More
☐	3 quantity	decimal(10,2)			No	None			 Change	 Drop  More
☐	4 unit	varchar(50)	utf8_unicode_ci		No	None			 Change	 Drop  More
☐	5 is_purchased	tinyint(1)			No	0			 Change	 Drop  More
☐	6 user_id 	int(11)			Yes	NULL			 Change	 Drop  More

Ingredients Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action	
☐	1 ingredient_id 	int(11)			No	None		AUTO_INCREMENT	 Change	 Drop  More
☐	2 ingredient_name 	varchar(255)	utf8_unicode_ci		No	None			 Change	 Drop  More
☐	3 default_unit	varchar(50)	utf8_unicode_ci		No	None			 Change	 Drop  More

Inventory Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action	
☐	1 inventory_id 	int(11)			No	None		AUTO_INCREMENT	 Change	 Drop  More
☐	2 ingredient_id 	int(11)			No	None			 Change	 Drop  More
☐	3 quantity	decimal(10,2)			No	None			 Change	 Drop  More
☐	4 unit	varchar(50)	utf8_unicode_ci		No	None			 Change	 Drop  More
☐	5 expiration_date	date			Yes	NULL			 Change	 Drop  More
☐	6 date_opened	date			Yes	NULL			 Change	 Drop  More
☐	7 user_id 	int(11)			Yes	NULL			 Change	 Drop  More

Nutritional_Values:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action	
☐	1 nutrition_id 	int(11)			No	None		AUTO_INCREMENT	 Change	 Drop  More
☐	2 ingredient_id 	int(11)			Yes	NULL			 Change	 Drop  More
☐	3 recipe_id 	int(11)			Yes	NULL			 Change	 Drop  More
☐	4 entity_type	enum('ingredient', 'recipe')	utf8_unicode_ci		No	None			 Change	 Drop  More
☐	5 calories	decimal(8,2)			Yes	NULL			 Change	 Drop  More
☐	6 protein	decimal(8,2)			Yes	NULL			 Change	 Drop  More
☐	7 carbs	decimal(8,2)			Yes	NULL			 Change	 Drop  More
☐	8 sugar	decimal(8,2)			Yes	NULL			 Change	 Drop  More
☐	9 fat	decimal(8,2)			Yes	NULL			 Change	 Drop  More

Recipes Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 recipe_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 user_id	int(11)			Yes	NULL			Change Drop More
☐	3 recipe_name	varchar(255)	utf8_unicode_ci		No	None			Change Drop More
☐	4 instructions	text	utf8_unicode_ci		Yes	NULL			Change Drop More
☐	5 image_url	varchar(500)	utf8_unicode_ci		Yes	NULL			Change Drop More
☐	6 source_api	varchar(100)	utf8_unicode_ci		Yes	NULL			Change Drop More
☐	7 last_fetched	timestamp			No	CURRENT_TIMESTAMP			Change Drop More
☐	8 cache_priority	enum('favorite', 'recently_used', 'temporary')	utf8_unicode_ci		No	None			Change Drop More
☐	9 serving_size	int(11)			Yes	1			Change Drop More

Recipe_Ingredients Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 recipe_id	int(11)			No	None			Change Drop More
☐	2 ingredient_id	int(11)			No	None			Change Drop More
☐	3 quantity	decimal(10,2)			No	None			Change Drop More
☐	4 unit	varchar(50)	utf8_unicode_ci		No	None			Change Drop More

Recipe_Tags Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 recipe_id	int(11)			No	None			Change Drop More
☐	2 tag_id	int(11)			No	None			Change Drop More
☐	3 date_tagged	timestamp			No	CURRENT_TIMESTAMP			Change Drop More

Tags Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 tag_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 tag_name	varchar(100)	utf8_unicode_ci		No	None			Change Drop More
☐	3 color	varchar(7)	utf8_unicode_ci		No	None			Change Drop More

Users Table:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 user_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 name	varchar(100)	utf8_unicode_ci		No	None			Change Drop More
☐	3 password	varchar(255)	utf8_unicode_ci		No	None			Change Drop More

Data Samples:

Users Data:

←T→				user_id	name	password
☐		Edit		Copy		Delete
1	liam	pass123				
☐		Edit		Copy		Delete
2	testuser	hashed_password_456				
☐		Edit		Copy		Delete
3	someuser	\$2y\$10\$1q\$1p/9P4c/iG0JX0tC7Aen2Ptn1ehYsa13XA7arm.j...				
☐		Edit		Copy		Delete
4	hungry	\$2y\$10\$Xt4l739DgUaVa2i6ZFI4wuCUMfpLSmZtZth0RLUuRI2...				
☐		Edit		Copy		Delete
5	megan	\$2y\$10\$0uZaVcJZ1Xa7CDEnTL/Cw0nV4oeUlp2Wh5qcdTe0bee...				
☐		Edit		Copy		Delete
6	Good_cook	\$2y\$10\$.2HRDvLD0wP3se5rvrygDevyyVLe80V2JIBTHR5ZuKA...				
☐		Edit		Copy		Delete
7	keelyf	\$2y\$10\$a/ptRXf/V/LWiW86xRoD00MriFhDG6eLa9.zmmhe6UW...				

Inventory Data:

←T→				inventory_id	ingredient_id	quantity	unit	expiration_date	date_opened	user_id
☐		Edit		Copy		Delete				
				3	4	200.00	grams	2025-04-01	2025-03-01	1
☐		Edit		Copy		Delete				
				5	7	300.00	mL	2025-11-01	2025-01-15	1
☐		Edit		Copy		Delete				
				6	8	3.00	count	NULL	NULL	1
☐		Edit		Copy		Delete				
				7	6	2.00	count	NULL	NULL	1
☐		Edit		Copy		Delete				
				9	10	500.00	grams	NULL	2026-04-29	1
☐		Edit		Copy		Delete				
				10	11	6.00	count	NULL	NULL	1
☐		Edit		Copy		Delete				
				14	17	8.00	ounces	2027-12-12	NULL	3
☐		Edit		Copy		Delete				
				16	18	2.00	lbs	NULL	NULL	1
☐		Edit		Copy		Delete				
				18	20	1.00	c	NULL	NULL	1
☐		Edit		Copy		Delete				
				19	29	2.00	cloves	NULL	NULL	1
☐		Edit		Copy		Delete				
				20	93	3.00	tablespoons	NULL	NULL	5
☐		Edit		Copy		Delete				
				21	92	4.00	tablespoons	NULL	NULL	5
☐		Edit		Copy		Delete				
				22	91	0.50	cup	NULL	NULL	5
☐		Edit		Copy		Delete				
				23	95	3.00	servings	NULL	NULL	5
☐		Edit		Copy		Delete				
				24	94	1.00	teaspoon	NULL	NULL	5
☐		Edit		Copy		Delete				
				25	43	2.00	cups	NULL	NULL	5
☐		Edit		Copy		Delete				
				26	14	2.00	tablespoons	NULL	NULL	1
☐		Edit		Copy		Delete				
				27	162	6.00	beans	2026-04-30	NULL	6
☐		Edit		Copy		Delete				
				28	69	15.00	ounce	NULL	NULL	6
☐		Edit		Copy		Delete				
				29	30	10.00	ounce	NULL	NULL	6
☐		Edit		Copy		Delete				
				30	70	2.00	tsp	NULL	NULL	6
☐		Edit		Copy		Delete				
				31	71	0.50	tsp	NULL	NULL	6
☐		Edit		Copy		Delete				
				32	51	0.25	tsp	NULL	NULL	6
☐		Edit		Copy		Delete				
				33	14	1.00	tsp	NULL	NULL	6
☐		Edit		Copy		Delete				
				34	35	0.25	cup	NULL	NULL	6

Recipes Data (Truncated):

		recipe_id	user_id	recipe_name	instructions	image_url	source_api	last_fetched	cache_priority	serving_size
			1	1 Spaghetti Carbonara	Boil pasta. Fry pancetta. Mix eggs and cheese. Com...	https://img.example.com/carbonara.jpg	Spoonacular	2026-02-22 06:22:03	favorite	1
			2	1 Chicken Stir Fry	Chop vegetables. Cook chicken. Add sauce and stir ...	https://img.example.com/stirfry.jpg	Spoonacular	2026-02-22 06:22:03	recently_used	1
			3	1 Avocado Toast	Toast bread. Mash avocado. Season and top with egg...	https://img.example.com/avotoast.jpg	Edamam	2026-02-22 06:22:03	temporary	1
			4	1 Beef Tacos	Brown beef. Season with spices. Fill tortillas wit...	https://img.example.com/tacos.jpg	Spoonacular	2026-02-22 06:22:03	favorite	1
			5	1 Greek Salad	Chop vegetables. Add olives and feta. Dress with o...	https://img.example.com/greeksalad.jpg	Edamam	2026-02-22 06:22:03	recently_used	1
			13	1 meatloaf	Step 1. Preheat oven to 375F Step 2. Mix all ingr...	NULL	User	2026-04-28 20:29:59		6
			14	1 Red Lentil Soup with Chicken and Turnips	1. To a large dutch oven or soup pot, heat the oil...	https://img.spoonacular.com/recipes/715415-556x378...	Spoonacular	2026-04-28 20:51:22		1
			15	1 Slow Cooker Beef Stew	1. To get started, heat your slow cooker to low. 2...	https://img.spoonacular.com/recipes/715446-556x378...	Spoonacular	2026-04-28 20:51:22		1
			16	1 Red Kidney Bean Jambalaya	1. Rinse the kidney beans and brown rice separatel...	https://img.spoonacular.com/recipes/782601-556x378...	Spoonacular	2026-04-28 20:51:22		1
			17	1 Quinoa and Chickpea Salad with Sun-Dried Tomatoes ...	1. Rinse the quinoa and soak for 8 hours or overni...	https://img.spoonacular.com/recipes/716004-556x378...	Spoonacular	2026-04-28 20:51:22		1
			18	1 Easy Homemade Rice and Beans	1. Heat the olive oil in a large pot over medium h...	https://img.spoonacular.com/recipes/716627-556x378...	Spoonacular	2026-04-28 20:51:22		1
			19	5 Green Monster Ice Pops	1. Place all ingredients into a blender and mix we...	https://img.spoonacular.com/recipes/645479-556x378...	Spoonacular	2026-04-29 09:35:36		1
			20	5 Caramel Peanut Fudge Cake	1. For the sponge, beat egg yolks with sugar for 3...	https://img.spoonacular.com/recipes/637016-556x378...	Spoonacular	2026-04-29 09:35:36		1
			21	5 Cacao chia pudding with avocado mousse	1. Add the chia seeds and psyllium in and slowly s...	https://img.spoonacular.com/recipes/636676-556x378...	Spoonacular	2026-04-29 09:35:36		1
			22	5 Chocolate Pudding - Rave Diet	1. Combine water, maple syrup, cocoa, cornstarch o...	https://img.spoonacular.com/recipes/639177-556x378...	Spoonacular	2026-04-29 09:35:36		1
			23	5 Fresh Cherry Scones	1. Cut cherries in half and pit them (or use a che...	https://img.spoonacular.com/recipes/643450-556x378...	Spoonacular	2026-04-29 09:35:36		1
			24	1 Baked Whole Wheat Breakfast Quesadilla	1. I know some of your kids have already returned ...	https://img.spoonacular.com/recipes/679540-556x378...	Spoonacular	2026-04-29 10:02:28		1
			25	1 Gujarati Dry Mung Bean Curry	1. Wash the mung beans and boil them in plenty of ...	https://img.spoonacular.com/recipes/646043-556x378...	Spoonacular	2026-04-29 10:02:28		1
			26	1 Slow Cooker Lamb Curry	1. Pull out your slow cooker and add everything in...	https://img.spoonacular.com/recipes/660290-556x378...	Spoonacular	2026-04-29 10:02:28		1
			27	1 Easy Cheesy Pizza Casserole	1. Brown ground beef in skillet; drain fat. 2. Mix...	https://img.spoonacular.com/recipes/641893-556x378...	Spoonacular	2026-04-29 10:02:28		1

D. Normalization

The database satisfies Third Normal Form across all tables. First Normal Form is satisfied because every column holds atomic values and each table has a defined primary key with no repeating groups. Second Normal Form is satisfied because every non-key attribute in each table depends on the full primary key rather than a partial key. Third Normal Form is satisfied because no non-key attribute depends on another non-key attribute.

A key design decision that supports 3NF is the separation of the Ingredients table from both Inventory and Recipe_Ingredients. Rather than storing ingredient names directly in inventory records or recipe ingredient lists, both tables reference ingredient_id as a foreign key. This means ingredient names and default units are stored exactly once, and any update to an ingredient name propagates automatically without risking inconsistency across the database.

Similarly, nutritional data is stored in its own Nutritional_Values table rather than embedded directly in Ingredients or Recipes. This avoids a transitive dependency where nutritional attributes would depend on ingredient or recipe names rather than their keys alone.

The use of junction tables for Recipe_Ingredients and Recipe_Tags also reflects a normalized design. Storing ingredient quantities or tag associations directly in the Recipes table would require repeating groups or comma-separated values, both of which violate First Normal Form. The junction tables resolve the many-to-many relationships cleanly while keeping each piece of data in exactly one place.

E. Application Functionality and Database Integration

SQL Queries

Query 1: Ingredients Expiring Within 30 Days

Purpose: This query retrieves all inventory items for a specific user that are at or within 30 days of their expiration date, along with the ingredient name and current quantity.

```
SELECT ing.ingredient_name, i.quantity, i.expiration_date
FROM Inventory i
JOIN Ingredients ing ON i.ingredient_id = ing.ingredient_id
WHERE i.user_id = 1 AND i.expiration_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY);
```

This query powers the expiration tracking feature of the application. It joins Inventory to Ingredients to retrieve human-readable names and uses DATE_ADD with CURDATE to calculate the 30-day window dynamically, so results are always relative to the current date. This is one of the more operationally useful queries in the application because it directly helps users reduce food waste.

```
1 SELECT ing.ingredient_name, i.quantity, i.expiration_date
2 FROM Inventory i
3 JOIN Ingredients ing ON i.ingredient_id = ing.ingredient_id
4 WHERE i.user_id = 1 AND i.expiration_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY);
```

Clear Format Get auto-saved query

☐ Bind parameters

Delimiter ; ☐ Show this query here again ☐ Retain query box ☐ Rollback when finished ☒ Enable foreign key c

Hide query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

✓ Showing rows 0 - 1 (2 total, Query took 0.0009 seconds.)

```
SELECT ing.ingredient_name, i.quantity, i.expiration_date FROM Inventory i JOIN Ingredients ing ON i.ingredient_id = ing.ingredi
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

ingredient_name	quantity	expiration_date
Parmesan Cheese	200.00	2025-04-01
Soy Sauce	300.00	2025-11-01

Query 2: User Favorites Sorted by Rating

Purpose: This query retrieves all recipes a user has favorited along with their rating and notes, sorted from highest to lowest rating.

```
SELECT r.recipe_name, f.rating, f.notes
FROM Favorites f
JOIN Recipes r ON f.recipe_id = r.recipe_id
WHERE f.user_id = 1 ORDER BY f.rating DESC;
```

This query supports the favorites tab of the application, where users can review recipes they have saved and rated. Sorting by rating descending means the user's highest-rated recipes appear at the top. The join between Favorites and Recipes retrieves the recipe name without duplicating it in the Favorites table. Currently, only one recipe is tagged favorite since we showcased the feature during the demo of favoriting and unfavoriting recipes.

The screenshot shows a SQL query editor interface. At the top, a text area contains the SQL query: `1 SELECT r.recipe_name, f.rating, f.notes`, `2 FROM Favorites f`, `3 JOIN Recipes r ON f.recipe_id = r.recipe_id`, and `4 WHERE f.user_id = 1 ORDER BY f.rating DESC;`. Below the text area are buttons for 'Clear', 'Format', and 'Get auto-saved query'. There is a checkbox for 'Bind parameters' with a help icon. Below that is a 'Delimiter' dropdown set to ';' and checkboxes for 'Show this query here again' and 'Retain query box'. A 'Hide query box' button is also present. A warning message states: 'Current selection does not contain a unique column. Grid edit, checkbox'. A green status bar indicates 'Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.) [rating: 5...'. Below this is a preview of the SQL query. There are links for 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', and 'Create PHP code'. At the bottom, there are controls for 'Show all', 'Number of rows' (set to 25), and a 'Filter rows' search box. An 'Extra options' button is also visible. The results table has three columns: 'recipe_name', 'rating' (with a dropdown set to 1), and 'notes'. The first row shows 'Spaghetti Carbonara' with a rating of 5 and the note '5 Family favorite, make every Sunday.'

```
1 SELECT r.recipe_name, f.rating, f.notes
2 FROM Favorites f
3 JOIN Recipes r ON f.recipe_id = r.recipe_id
4 WHERE f.user_id = 1 ORDER BY f.rating DESC;
```

Clear Format Get auto-saved query

☐ Bind parameters ?

Delimiter ; ☐ Show this query here again ☐ Retain query box

Hide query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.) [rating: 5...

```
SELECT r.recipe_name, f.rating, f.notes FROM Favorites f JOIN Recipes r ON
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code]

☐ Show all | Number of rows: 25 Filter rows: Search this tab

Extra options

recipe_name	rating ▾ 1	notes
Spaghetti Carbonara	5	5 Family favorite, make every Sunday.

Query 3: Tag Summary with Average Ratings

Purpose: This query returns each tag alongside a count of how many distinct recipes carry that tag and the average rating of those recipes across all users.

```
SELECT t.tag_name, COUNT(DISTINCT rt.recipe_id) AS recipe_count, AVG(f.rating) AS
avg_rating
FROM Tags t JOIN Recipe_Tags rt ON t.tag_id = rt.tag_id
LEFT JOIN Favorites f ON rt.recipe_id = f.recipe_id
GROUP BY t.tag_id, t.tag_name;
```

This query provides an aggregate view of recipe organization by tag. The LEFT JOIN on Favorites means tags with no associated ratings still appear in the results with a null average rather than being excluded. Grouping by both tag_id and tag_name ensures correctness and allows the results to display the tag name without ambiguity. Some recipes that were tagged during the demo have no rating.

The screenshot displays a SQL query editor interface. At the top, the SQL query is entered: `SELECT t.tag_name, COUNT(DISTINCT rt.recipe_id) AS recipe_count, AVG(f.rating) AS avg_rating FROM Tags t JOIN Recipe_Tags rt ON t.tag_id = rt.tag_id LEFT JOIN Favorites f ON rt.recipe_id = f.recipe_id GROUP BY t.tag_id, t.tag_name;`. Below the query editor are buttons for 'Clear', 'Format', and 'Get auto-saved query', along with a checkbox for 'Bind parameters'. A status bar indicates the delimiter is semicolon and provides options to show the query again, retain the query box, or rollback when finished. A 'Hide query box' button is also present. A warning message states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete functions are disabled.' Below this, a green banner indicates 'Showing rows 0 - 6 (7 total, Query took 0.0005 seconds.)'. The query is repeated above a table of results. The table has columns 'tag_name', 'recipe_count', and 'avg_rating'. The results are as follows:

tag_name	recipe_count	avg_rating
Italian	1	5.0000
Quick	2	NULL
High Protein	3	5.0000
Vegetarian	1	NULL
Mexican	1	NULL
Sunday Meal	1	NULL
tlll	1	NULL

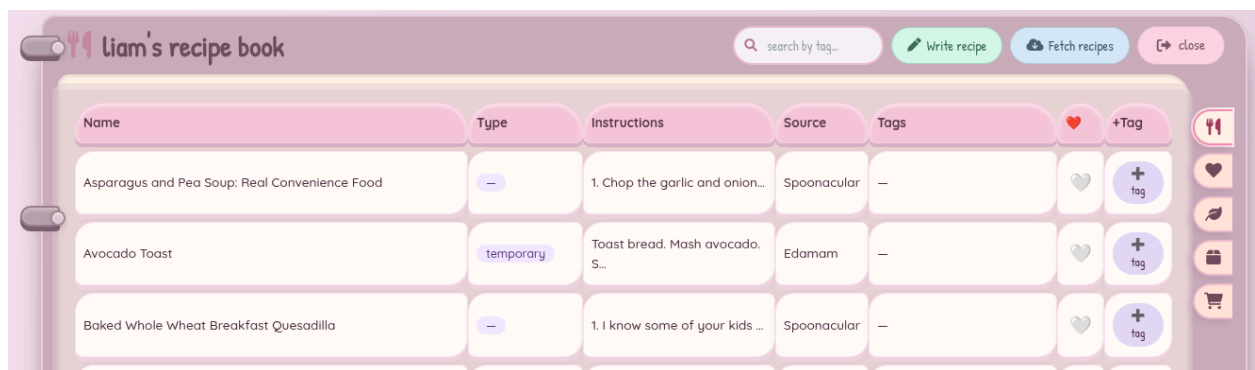
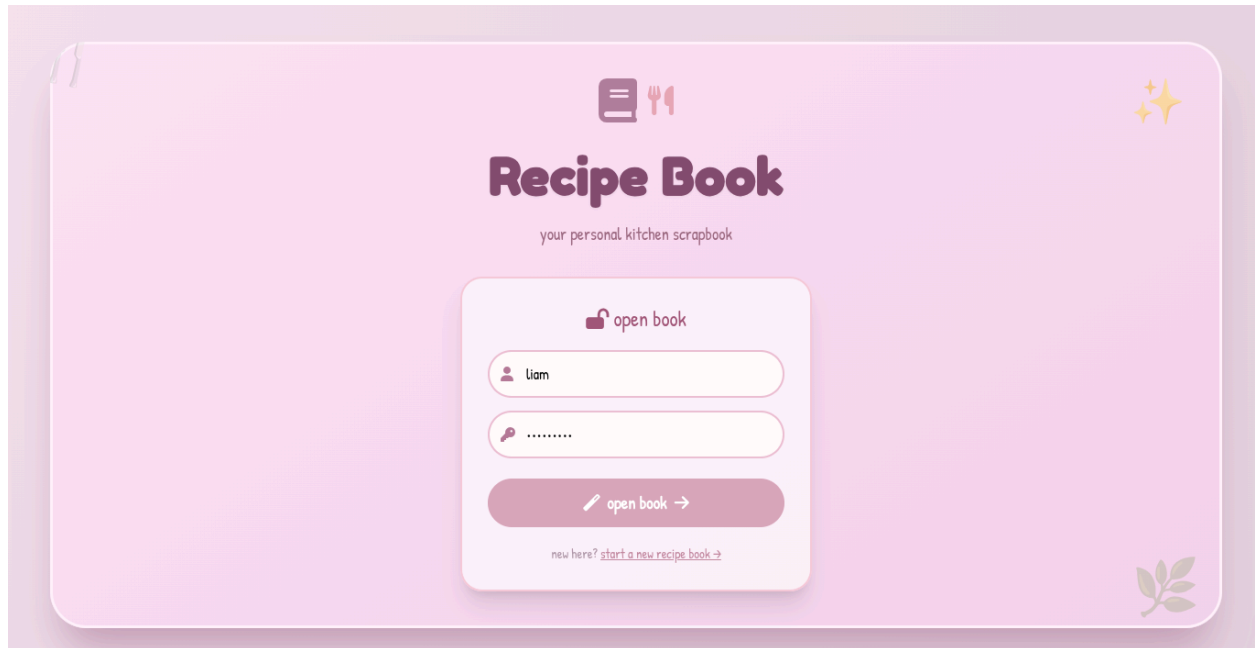
Below the table, there are controls for 'Show all', 'Number of rows' (set to 25), and a 'Filter rows' search box. An 'Extra options' button is also visible.

Application Interface

The application has a clean, consistent layout with a navigation bar that gives users access to all major sections: inventory, recipes, grocery list, favorites, tags, and nutrition. The interface uses a pink and neutral color palette applied consistently across pages, giving the application a polished and recognizable visual identity.

Each section is accessible with a single click from the navigation bar. Pages are organized to surface the most relevant information immediately, such as showing expiring items prominently in the inventory view and displaying recipe cards with images in the recipe browser. Forms for adding inventory items, recipes, and grocery list entries are straightforward and include appropriate input validation.

The overall user experience prioritizes simplicity. A user can log in, check what is expiring, browse or search for a recipe, and add missing ingredients to their grocery list without navigating through multiple layers of menus. The application is designed to complete these common tasks with minimal friction.



Frontend and Backend Integration

The application is built with HTML, CSS, and JavaScript on the frontend and PHP on the backend, with MySQL as the database. Pages are served through cPanel and make PHP requests to query or update the database based on user actions.

When a user adds an ingredient to their inventory, a form submission triggers a PHP script that executes an INSERT statement into the Inventory table with the supplied values. When a user marks grocery items as purchased, the frontend sends the relevant user ID to a PHP handler that runs the UPDATE query described above. Recipe data fetched from the Spoonacular API is parsed in PHP and inserted into the Recipes and Recipe_Ingredients tables before being displayed to the user.

All database interactions go through PHP scripts that handle query construction, execution, and result formatting. Results are returned to the frontend as HTML or JSON depending on the context and rendered dynamically on the page.

Liam's recipe book

search by tag... Write recipe Fetch recipes close

Ingredient	Qty	Unit	Expiration	Opened
Avocado	3.00	count	—	set opened remove
Bell Pepper				set opened remove
Ground Beef			2026-04-29	set opened remove
Tortillas				set opened remove
Ground beef (80/20)				set opened remove
breadcrumbs				set opened remove
garlic				set opened remove
Olive Oil				set opened remove
beef broth				set opened remove
carrots				set opened remove
cream of mushroom soup				set opened remove
onion				set opened remove
celery				set opened remove
equivalent amount of a ground beef/bulk sausage				set opened remove
Parmesan Cheese			2025-03-01	set opened remove
Soy Sauce	300.00	ml	2025-11-01	2025-01-15 remove

Write a Recipe

your own creation

Recipe Name
e.g. Grandma's Lasagna

Type
Dinner

Serving Size
4

Instructions
Step 1. Preheat oven to 375°F
Step 2. ...

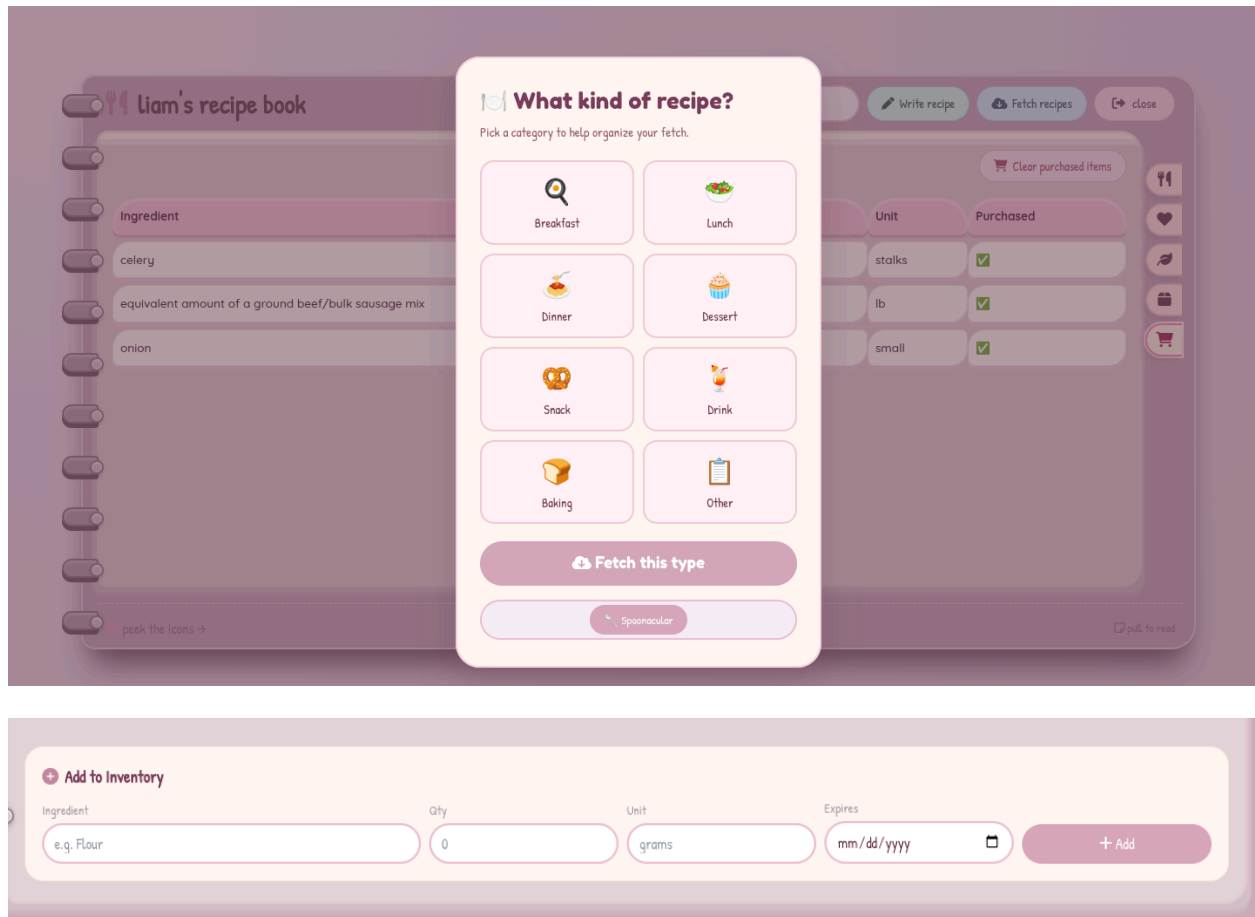
Ingredients (one per line: "2 cups flour")
2 cups flour
1 tsp salt
3 eggs

Save Recipe

Add to Inventory

Ingredient Qty Unit Expires

e.g. Flour 0 grams mm/dd/yyyy + Add



F. Security, Privacy and Data Protection

Security was addressed at multiple layers of the application to protect user data and prevent common web vulnerabilities.

All database queries that incorporate user input are executed using prepared statements. Prepared statements separate the SQL query structure from the data values supplied at runtime, which prevents SQL injection attacks by ensuring user input is never interpreted as part of the query itself.

Output displayed to users is sanitized before rendering to prevent cross-site scripting attacks. Any data retrieved from the database that is output to the browser is passed through appropriate escaping functions to neutralize potentially malicious HTML or JavaScript that could have been stored in the database.

User passwords are never stored in plain text. When a user registers, their password is hashed using PHP's password_hash function before being written to the Users table. Authentication uses password_verify to compare the submitted password against the stored hash without ever reconstructing or exposing the original value.

The application enforces authentication on all protected pages. Users must log in before accessing inventory, recipes, grocery lists, or any other personal data. Session management is handled server-side, and unauthenticated requests are redirected to the login page. Users after the first two (basic function test accounts) have their passwords encrypted using bcrypt:

		user_id	name	password
☐	Edit Copy Delete	1	liam	pass123
☐	Edit Copy Delete	2	testuser	hashed_password_456
☐	Edit Copy Delete	3	someuser	\$2y\$10\$1qS1p/9P4c/i60JX0tC7Aen2Ptn1ehYsa13XA7arm.j...
☐	Edit Copy Delete	4	hungry	\$2y\$10\$Xt4L739DgUaVa2i6ZFI4wuCUMfpLSmZtZth0RLUuRI2...
☐	Edit Copy Delete	5	megan	\$2y\$10\$0uZaVcJZ1Xa7CDEnTL/Cw0nV4oeUlp2Wh5qcdTe0bee...
☐	Edit Copy Delete	6	Good_cook	\$2y\$10\$.2HRDvLD0wP3se5rvrygDevyyVLe80V2JIBTHR5ZuKA...
☐	Edit Copy Delete	7	keelyf	\$2y\$10\$a/ptRXf/V/LWiW86xRoD00MriFhDG6eLa9.zmmhe6UW...

G. Optimization and Reliability Enhancements

Indexed Columns

Indexes were added to foreign key columns that are frequently used in JOIN operations and WHERE clause filters. Specifically, indexes on `user_id` in the Inventory, Grocery_List, Favorites, and Recipes tables improve the performance of queries that filter or join on a specific user. Without these indexes, queries that retrieve a user's inventory or recipe list would require a full table scan, which becomes increasingly expensive as the number of records grows.

An index on `ingredient_id` in the Inventory and Recipe_Ingredients tables similarly improves JOIN performance when resolving ingredient names from the Ingredients reference table.

Grocery_List Table Indexing:

Indexes									
Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	list_id	18	A	No	
Edit Rename Drop	ingredient_id	BTREE	No	No	ingredient_id	16	A	No	
Edit Rename Drop	user_id	BTREE	No	No	user_id	3	A	Yes	

Ingredients Table Indexing:

Indexes									
Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	inventory_id	34	A	No	
Edit Rename Drop	ingredient_id	BTREE	No	No	ingredient_id	31	A	No	
Edit Rename Drop	user_id	BTREE	No	No	user_id	4	A	Yes	

Recipe_Ingredients Table Indexing:

Indexes									
Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	recipe_id	35	A	No	
					ingredient_id	351	A	No	
Edit Rename Drop	ingredient_id	BTREE	No	No	ingredient_id	209	A	No	

Query Optimization

The expiration tracking query was identified as a candidate for optimization given that it runs frequently and filters across potentially large inventory datasets. The query joins Inventory to Ingredients and filters by both user_id and a date range computed at runtime.

With indexes on user_id and expiration_date in the Inventory table, the database engine can narrow the result set to a specific user and then apply the date filter without scanning unrelated rows. Using DATE_ADD with CURDATE inside the query rather than computing the date in application code keeps the filtering entirely within the database engine, reducing data transfer between the application and the database.

Run SQL query/queries on database liammcke_inventory_recipes: ⓘ

```
1 EXPLAIN SELECT ing.ingredient_name, i.quantity, i.expiration_date
2 FROM Inventory i
3 JOIN Ingredients ing ON i.ingredient_id = ing.ingredient_id
4 WHERE i.user_id = 1 AND i.expiration_date < DATE_ADD(CURDATE(), INTERVAL 30 DAY);
```

Clear Format Get auto-saved query

☐ Bind parameters ⓘ

Delimiter ; ☐ Show this query here again ☐ Retain query box ☐ Rollback when finished ☒ Enable foreign key checks Go

Hide query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

Your SQL query has been executed successfully.

```
EXPLAIN SELECT ing.ingredient_name, i.quantity, i.expiration_date FROM Inventory i JOIN Ingredients ing ON i.ingredient_id = ing.ingredient_id WHERE i.user_id = 1 AND i.expiration_date < DATE_ADD(CURDATE(), INTERVAL 30 DAY);
```

[Edit inline] [Edit] [Skip Explain SQL] [Create PHP code]

Extra options

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	i	NULL	ref	ingredient_id,user_id	user_id	5	const	16	33.33	Using where
1	SIMPLE	ing	NULL	eq_ref	PRIMARY	PRIMARY	4	liammcke_inventory_recipes.i.ingredient_id	1	100.00	NULL

Transactions

Adding a recipe from the Spoonacular API involves multiple dependent write operations: inserting a record into the Recipes table and then inserting one or more records into Recipe_Ingredients for each ingredient associated with that recipe. These operations are wrapped in a transaction to preserve data integrity.

If the Recipes insert succeeds but a subsequent Recipe_Ingredients insert fails, the transaction is rolled back, leaving the database in its original state. This prevents orphaned recipe records that reference ingredients which were never stored, which would cause errors in any query that joins the two tables. Wrapping these operations in a transaction ensures the database is always consistent regardless of whether the full operation completes successfully.

```
if (!$exists) {
    try {
        $pdo->beginTransaction();

        pdo($pdo, '
            INSERT INTO Recipes
            (user_id, recipe_name, instructions, image_url, source_api, cache_priority, last_fetched)
            VALUES (?, ?, ?, ?, ?, ?, NOW())
        ', [$user_id, $name, $instructions, $image_url, 'Spoonacular', $type]);
        $recipe_id = (int)$pdo->lastInsertId();

        foreach ($recipe['extendedIngredients'] ?? [] as $ing) {
            $ing_name = sanitize_str($ing['name'] ?? '', 100);
            $qty = (float)$ing['amount'] ?? 0;
            $unit = sanitize_str($ing['unit'] ?? 'count', 50);
            if (!$ing_name) continue;

            $sex = pdo($pdo, 'SELECT ingredient_id FROM Ingredients WHERE LOWER(ingredient_name) = LOWER(?) LIMIT 1', [$ing_name])->fetch();
            if ($sex) { $ing_id = (int)$sex['ingredient_id']; }
            else { pdo($pdo, 'INSERT INTO Ingredients (ingredient_name, default_unit) VALUES (?, ?)', [$ing_name, $unit]); $ing_id = (int)$pdo->lastInsertId(); }

            pdo($pdo, 'INSERT IGNORE INTO Recipe_Ingredients (recipe_id, ingredient_id, quantity, unit) VALUES (?, ?, ?, ?)',
                [$recipe_id, $ing_id, $qty, $unit]);
        }

        $pdo->commit();
        $added++;
    } catch (Exception $e) {
        $pdo->rollBack();
        $errors[] = 'Transaction failed for recipe: ' . $name;
    }
}
```

H. Reflection and Final Improvements

Strengths

Feedback from the Database Expo consistently highlighted the polish and interactivity of the interface, along with the amount of features the application offers. Reviewers called out recipe tagging, external recipe fetching, grocery list management, favorites, and expiration tracking as standout elements. Password hashing was also noted as a clear security strength.

We agree with this assessment. The feature set came together cohesively, and the connections between inventory, recipes, favorites, tags, grocery lists, and expiration tracking made the application feel complete rather than like a collection of isolated pieces. The database structure supported this well, keeping relationships clean and making it straightforward to build and link each part of the system. Security was handled correctly from the start, and the interface made all of it accessible without requiring any technical knowledge from the user.

Revisions and Improvements

Based on feedback received at the Database Expo, we identified one primary change to make before the final submission. The Edamam API integration was removed from the application. During the expo, this integration caused instability that disrupted the demonstration and was the likely source of one reviewer's comment that not everything appeared fully tested. Removing Edamam eliminates the unreliable dependency while leaving the Spoonacular integration, which functions correctly, in place.

The nutrition tab remains in the application and displays nutritional data stored in the database. Full nutrition lookup through an external API was not implemented because the relevant Spoonacular endpoints are behind a paid tier. The tab represents the foundation of what that feature would look like if paid API access were available.

We did not change the background color or color palette in response to one reviewer's comment. Reworking the full visual palette across the application was out of scope for this revision phase and would have introduced unnecessary risk to a working and visually consistent interface. We also did not implement a frequently purchased items recommendation feature, as the database design and application logic required would have been too significant to complete without compromising existing functionality.

Team Contributions

Development responsibilities for the application were divided across the team. Liam handled backend development and coordinated the overall project effort. Kat built the frontend, with design input from Nick and Nathan.

Lessons Learned

One of the biggest challenges was connecting several moving parts into one stable application, including user accounts, inventory tracking, recipes, tags, grocery lists, nutrition data, and API-based recipe importing. We also ran into issues with external API reliability, especially with the Edamam API, which caused instability during our demo and was later removed so the working Spoonacular integration could remain. Another challenge was making sure the database stayed secure and reliable through password hashing, input validation, and proper foreign key relationships.

Building this database application showed us how important it is to design the database carefully before adding features to a working application since normalized tables, junction tables, and foreign keys made the recipe, inventory, tag, and grocery systems work together cleanly. We also learned that a successful database application goes beyond simply storing data. It is important to make that data useful through a clean interface, secure accounts, optimized queries, and useful features that solve real problems, such as reducing food waste.

Future Improvements

The most immediate future improvement would be upgrading to a paid Spoonacular tier to unlock nutrition data lookup. This would allow the nutrition tab to display live macronutrient data pulled directly from the API rather than relying on manually populated database records, significantly expanding the usefulness of that feature.

A frequently purchased items recommendation feature would also be a natural extension of the existing grocery list and inventory data. By analyzing purchase history and inventory patterns, the application could suggest items a user likely needs without them having to add entries manually each time.

On the technical side, adding pagination to recipe and inventory lists would improve performance and usability as a user's data grows. The current implementation retrieves all records for a given user in a single query, which works well for moderate data sizes but would benefit from pagination at scale.